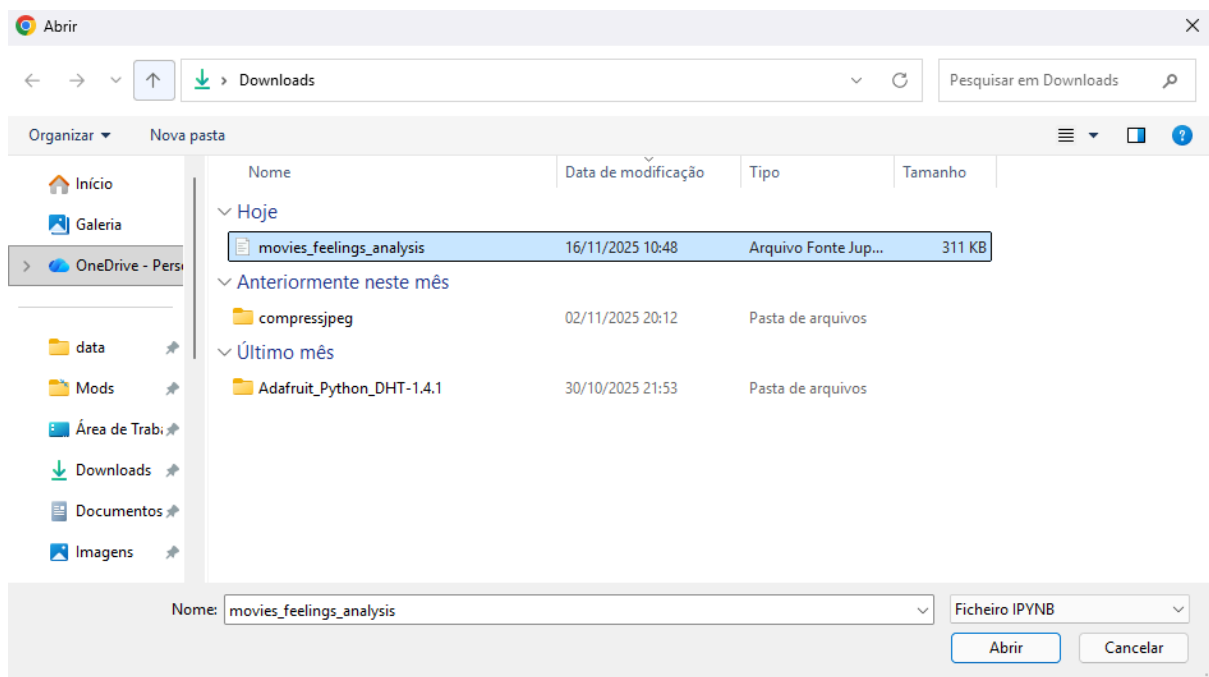
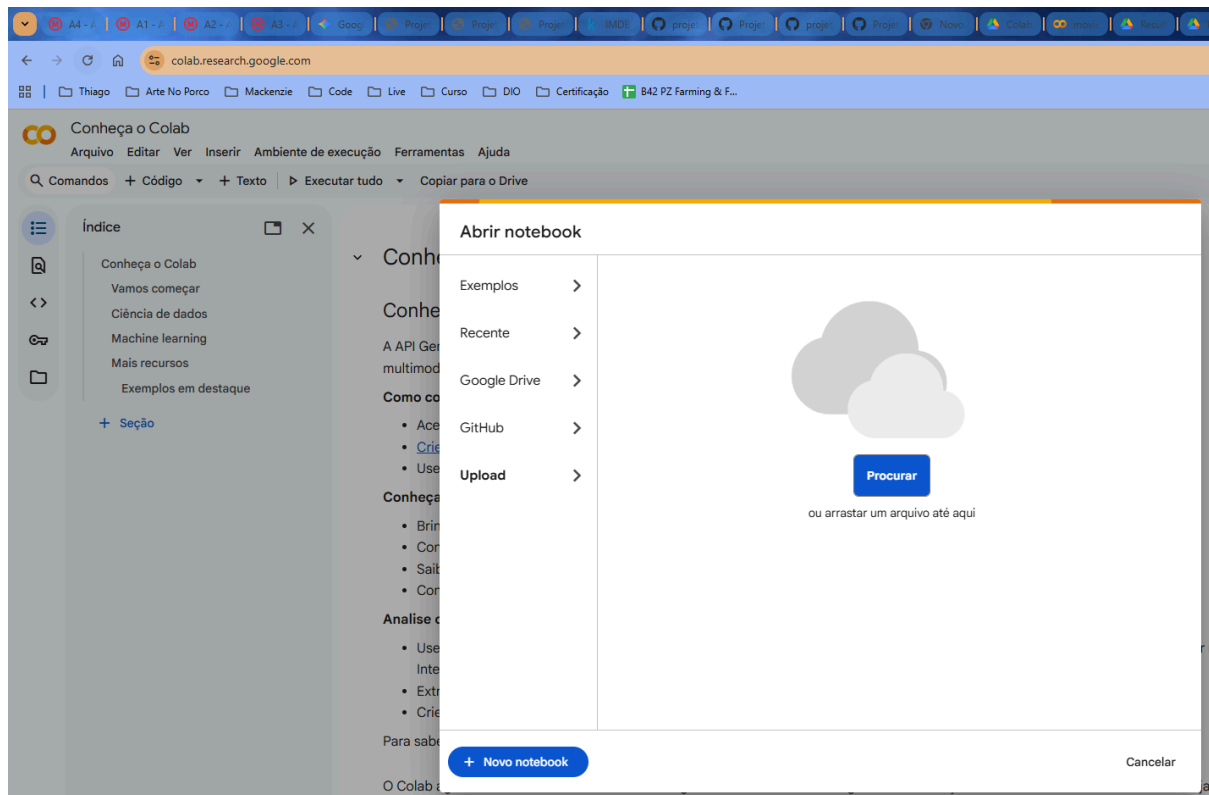
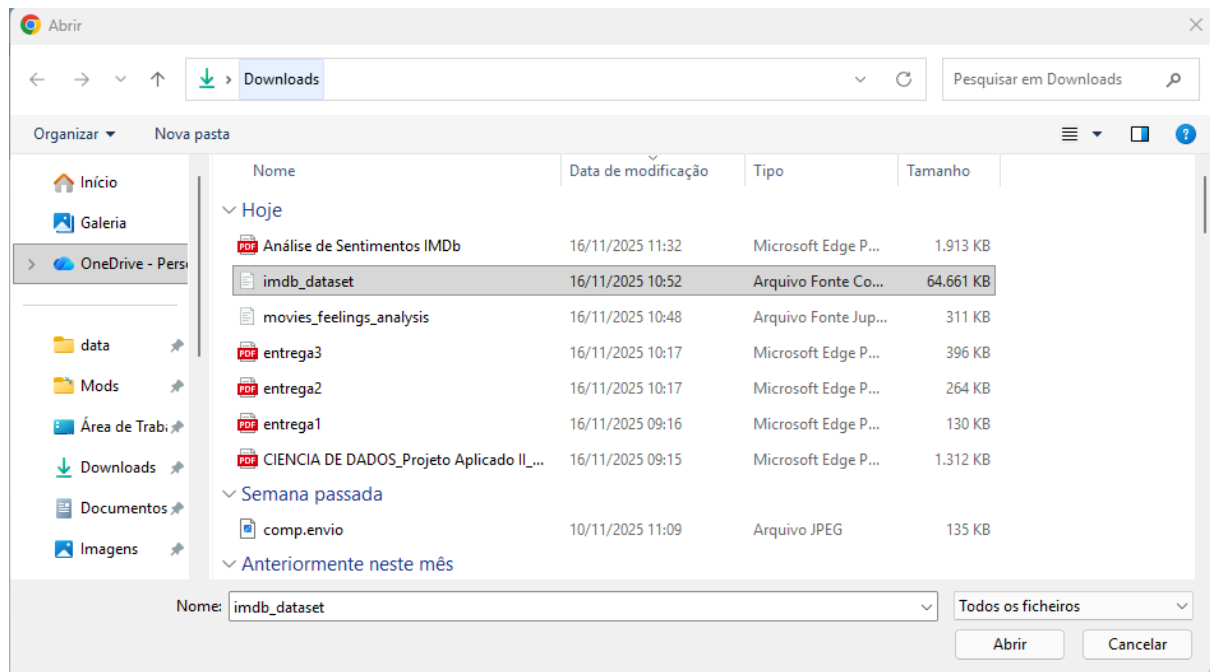


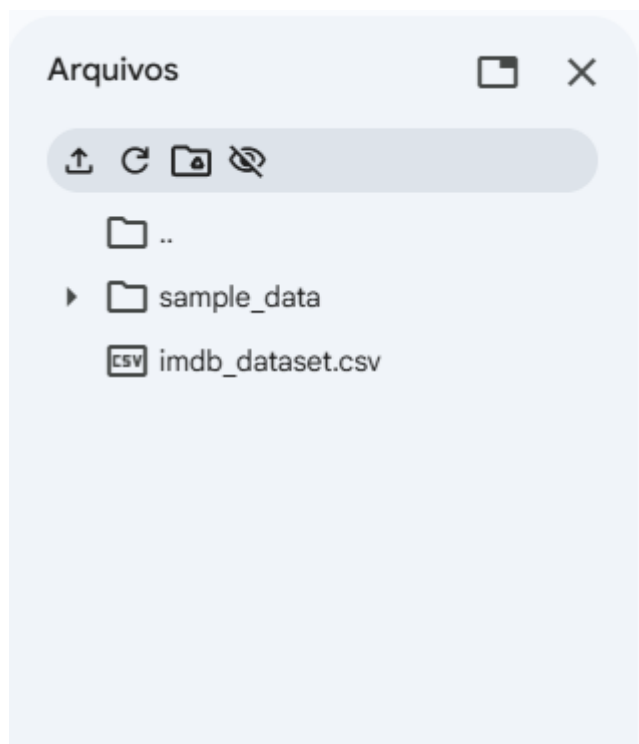
Para executar o código, podemos acessar o Colab (<https://colab.research.google.com/>), fazer o login, de preferência com uma conta google. Com o isso podemos fazer o upload do nosso código (.ipynb).



Ao iniciar o notebook, aproveite e importe a nossa base de dados, aqui no caso oriunda do kaggle, o arquivo é delimitado com o formato de CSV.



Após o upload do arquivo, teremos os seguintes arquivos disponíveis no nosso notebook.



A partir deste momento podemos começar a executar os blocos de códigos do notebook.

A 1 Etapa, diz respeito a configurações do setup e import das bibliotecas.
Ao executar será necessário dar acesso ao notebook acessar as pastas do Google Drive.

✓ Movies feelings analysis

✓ 1) ETAPA

Importando bibliotecas e o Setup

[1]

✓ 46s

```
import os, re, random
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
import nltk; nltk.download('stopwords')
from nltk.corpus import stopwords
from google.colab import drive
drive.mount('/content/drive')
sns.set_theme() # estilo visual do seaborn

RANDOM_STATE = 42
random.seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)
```

✓ ... [nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
Mounted at /content/drive

Permitir que este notebook acesse seus arquivos do Google Drive?

Este notebook está solicitando acesso aos seus arquivos do Google Drive.
Conceder isso permite que o código executado neste notebook modifique arquivos no Drive. Confira o código do notebook antes de permitir o acesso.

Não

[Conectar ao Google Drive](#)

Etapa 2 trás a criação da pasta a onde será retornado os dados que irão ser gerados nas execuções dos códigos do notebook.

✓ 2) ETAPA

Cria a pasta para salvar saídas

[2]

✓ Os



```
os.makedirs('reports', exist_ok=True)
```

A Etapa 3 irá carregar o nosso arquivo em CSV para dentro do nosso código para poder utilizar os dados nas nossas análises.

✓ 3) ETAPA

Carregar dados

[3]

✓ Os



```
df = pd.read_csv('/content/imdb_dataset.csv')  
df = df.rename(columns={'review': 'text', 'sentiment': 'label'})
```

Etapa 4 começamos a analisar os dados que acabamos de importar, Informação do type dos campos que temos, e também o retorno das 3 primeiras linhas.

4) ETAPA

Sanidade básica

```
|  
Js  
▶ print("Formato:", df.shape)  
  print(df.head(3))  
  print("Nulos por coluna:\n", df.isna().sum())
```

```
... Formato: (50000, 2)
```

	text	label
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive

```
Nulos por coluna:  
  text      0  
  label     0  
dtype: int64
```

A 5 Etapa identificamos os dados duplicados, por expressões exatas, obtemos 824 registros.

Imprimimos 10 linhas como amostras.

5) ETAPA

Procuramos por duplicatas exatas (texto, label) e imprimimos alguns exemplos

```
before = len(df)
dups_exact = df.duplicated(subset=['text', 'label'], keep=False)
print("Duplicatas exatas (text+label):", dups_exact.sum())

# Amostra de 5 pares (mesmo text+label)
ex_dups = (df.loc[dups_exact, ['text', 'label']]
           .groupby(['text', 'label'])
           .head(2) # mostra duas ocorrências
           .head(10)) # limita linhas exibidas

# Trunca os textos para visualizar melhor
ex_dups_display = ex_dups.copy()
ex_dups_display['text'] = ex_dups_display['text'].str.slice(0, 220)
ex_dups_display
```

... Duplicatas exatas (text+label): 824

	text	label	
42	Of all the films I have seen, this one, The Ra...	negative	
84	We brought this film as a joke for a friend, a...	negative	
140	Before I begin, let me get something off my ch...	negative	
219	Ed Wood rides again. The fact that this movie ...	negative	
245	I have seen this film at least 100 times and I...	positive	
480	From director Barbet Schroder (Reversal of For...	negative	
513	The story and the show were good, but it was r...	negative	
636	I rented this thinking it would be pretty good...	negative	
638	This movie has everything typical horror movie...	positive	
701	I Enjoyed Watching This Well Acted Movie Very ...	positive	

Etapa 6 tratamos os casos que temos duplicados, removendo.

Auditamos retornos dos conflitos, validando a quantidade de retornos, removido 418 registros.

E não foi identificado nenhum rótulo com conflito.

6) ETAPA

Removemos duplicatas exatas por (text, label) para evitar contagem dupla de exemplos idênticos.

Em seguida, auditamos textos repetidos com rótulos conflitantes, apenas reportando sua frequência para decisão futura.

O processo foi realizado antes da divisão treino/validação para mitigar vazamento de dados.

```
before = len(df)

# Remover duplicatas exatas (texto + rótulo)
df = df.drop_duplicates(subset=['text', 'label']).reset_index(drop=True)

# Auditar conflitos de rótulo para o mesmo texto
conflict_mask = df.duplicated(subset=['text'], keep=False)
conflicts = (df.loc[conflict_mask]
             .groupby('text')['label']
             .nunique()
             .reset_index(name='n_labels'))
n_conflicts = (conflicts['n_labels'] > 1).sum()

after = len(df)
print(f"Removidas {before - after} duplicatas exatas (texto+label).")
print(f"Textos com rótulos conflitantes: {n_conflicts}")

... Removidas 418 duplicatas exatas (texto+label).
    Textos com rótulos conflitantes: 0
```

Identificamos 824 linhas pertencentes a 406 grupos de duplicatas exatas por (text,label). Mantivemos uma ocorrência por grupo e removemos 418 linhas duplicadas, antes do split de treino/validação (alguns grupos tinham mais de duas ocorrências).

Etapa 7 iremos tratar os texto buscando por expressões específicas, e também por quebras de linhas.

Imprimimos o retorno com ocorrência específica do texto. Retornando 0 ocorrência.

7) ETAPA

Tratamento dos dados, como remoção de espaços, e textos de quebras

Gravação dos dados tratados para futura consultas

```
# remover <br>, <br/>, <br /> e colapsar espaços
df['text'] = df['text'].str.replace(r'<br\s*/?>', ' ', regex=True)
df['text'] = df['text'].str.replace(r'\s+', ' ', regex=True).str.strip()

# checando se sobrou alguma coisa
restantes = df['text'].str.contains(r'<br\s*/?>', regex=True).sum()
print("Ocorrências de <br> restantes:", restantes)

... Ocorrências de <br> restantes: 0
```

Avaliamos se o campo 'text' não é uma string

Avaliamos se o campo 'label' tem apenas o retorno de "positive" e "negative".

Imprimimos o resultado que temos, 49.582 registros totais com uma distribuição de 24.884 registros como positive e 24.698 registro como negative.

Assim obtemos os dados limpos e tratados, podendo ser salvo na nossa pasta data-processed, com o arquivo imdb_clean.csv.

Imprimimos os resultados:

```
# Schema simples (contrato de dados)
assert df['text'].map(lambda x: isinstance(x, str)).all(), "Há linhas sem texto (não-string)."
assert set(df['label'].unique()) <= {'positive', 'negative'}, "Labels fora do conjunto esperado."

# Confirmação (só é executado se as verificações acima forem verdadeiras)
print(
    "OK: schema verificado – coluna 'text' é string e 'label' contém apenas {'positive', 'negative'}.\n"
    f"Registros: {len(df)} | Distribuição de classes: {df['label'].value_counts().to_dict()}"
)

# Salvar dataset limpo
CSV_OUT = '/content/data-processed/imdb_clean.csv'
os.makedirs('/content/data-processed', exist_ok=True)

df_clean = df[['text', 'label']].copy()
df_clean.to_csv(CSV_OUT, index=False)

# feedback útil
counts = df_clean['label'].value_counts().to_dict()
print(f"Salvo: {CSV_OUT} | Registros: {len(df_clean)} | Distribuição: {counts}")

*** OK: schema verificado – coluna 'text' é string e 'label' contém apenas {'positive', 'negative'}.
Registros: 49582 | Distribuição de classes: {'positive': 24884, 'negative': 24698}
Salvo: /content/data-processed/imdb_clean.csv | Registros: 49582 | Distribuição: {'positive': 24884, 'negative': 24698}
```


Continuando com as nossas análises, avaliamos nossa distribuição das classes e adicionamos nossa apresentação os gráficos.

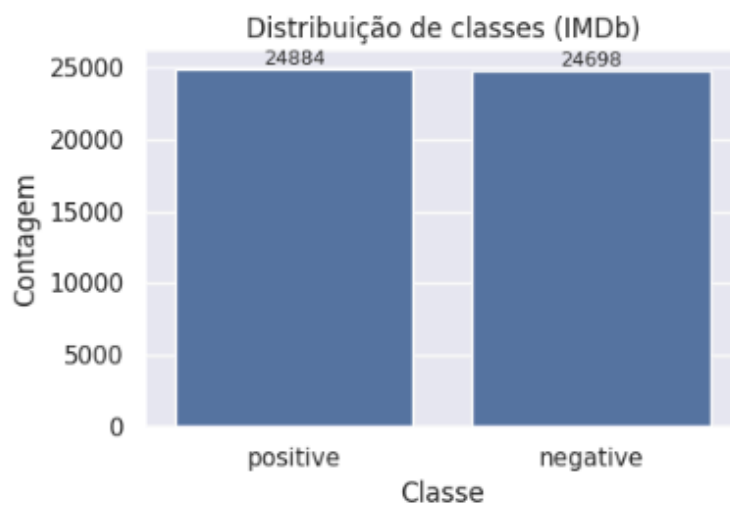
8) ETAPA

Distribuição e entendimento dos dados por classes, e apresentação por graficos.

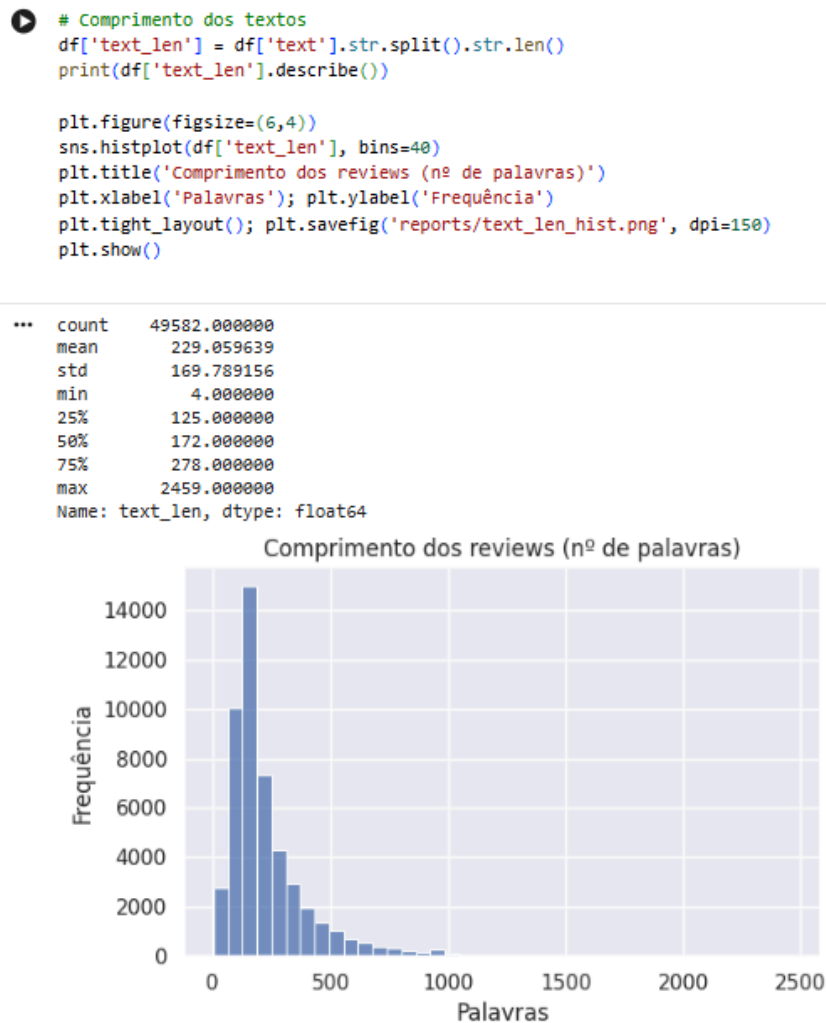
```
counts = (df['label']
          .value_counts()
          .rename_axis('label')
          .reset_index(name='count'))
counts['pct'] = (counts['count'] / counts['count'].sum() * 100).round(2)
print(counts)

plt.figure(figsize=(5,3.5))
ax = sns.barplot(data=counts, x='label', y='count')
ax.set_title('Distribuição de classes (IMDb)')
ax.set_xlabel('Classe'); ax.set_ylabel('Contagem')
for p in ax.patches:
    ax.annotate(f"{p.get_height():.0f}",
               (p.get_x()+p.get_width()/2, p.get_height()),
               ha='center', va='bottom', fontsize=9)
plt.tight_layout(); plt.savefig('reports/class_balance.png', dpi=150)
plt.show()
```

```
...      label  count  pct
0  positive  24884  50.19
1  negative  24698  49.81
```



Medimos o comprimento dos textos e também a ocorrência de repetição dos textos com o mesmo comprimento, exibimos um gráfico que nós mostra um retorno de comprimento de texto de 4 até 2459 palavras, com uma média de 229 palavras.



Imprimimos algumas amostras.

```
# Exemplos de linhas (1 positiva, 1 negativa, 1 mediano por comprimento) ==
ex_pos = df[df['label']=='positive'].sample(1, random_state=RANDOM_STATE)['text'].iloc[0]
ex_neg = df[df['label']=='negative'].sample(1, random_state=RANDOM_STATE)['text'].iloc[0]
mid_idx = (df['text_len'] - df['text_len'].median()).abs().sort_values().index[0]
ex_mid = df.loc[mid_idx, 'text']

print("\nEXEMPLO POSITIVO:\n", ex_pos[:140], "\n")
print("EXEMPLO NEGATIVO:\n", ex_neg[:140], "\n")
print("EXEMPLO MEDIANO POR COMPRIMENTO:\n", ex_mid[:140], "\n")
```

```
***
EXEMPLO POSITIVO:
Not since The Simpsons made it's debut has there been a sitcom that I didn't want to turn of in a matter of 2 minutes. It has of course been

EXEMPLO NEGATIVO:
I usually much prefer French movies over American ones, with explosions and car chases, but this movie was very disappointing. There is no w

EXEMPLO MEDIANO POR COMPRIMENTO:
I was looking forward to seeing this movie after reading a positive review in the New York Times. In addition, I'm also Shanghainese so ther
```

Fazemos a limpeza nos textos sobre URLs, Links, tratamos de apenas considerar letras e espaços.

E fazemos uma impressão dos termos mais usados em resultados positivos e ou negativos, gravamos também as amostragens, dentro da nossa pasta de reports.

```
# Top termos por classe (limpeza simples)|
stop_en = set(stopwords.words('english'))

def basic_clean(s: str):
    s = s.lower()
    s = re.sub(r'http\S+|www\S+', ' ', s)      # URLs
    s = re.sub(r'^a-z\s', ' ', s)             # mantém só letras e espaço
    toks = [w for w in s.split() if w not in stop_en and len(w) > 2]
    return toks

from collections import Counter

def top_words(subdf, k=15, cap=20000):
    c = Counter()
    # limitar a cap linhas p/ rapidez (ajuste se quiser)
    for t in subdf['text'].head(cap):
        c.update(basic_clean(t))
    top = pd.DataFrame(c.most_common(k), columns=['term', 'count'])
    return top

top_pos = top_words(df[df['label']=='positive'], k=15)
top_neg = top_words(df[df['label']=='negative'], k=15)

top_pos.to_csv('reports/top_terms_positive.csv', index=False)
top_neg.to_csv('reports/top_terms_negative.csv', index=False)
print("\nTop termos (positive):\n", top_pos.head())
print("\nTop termos (negative):\n", top_neg.head())
```

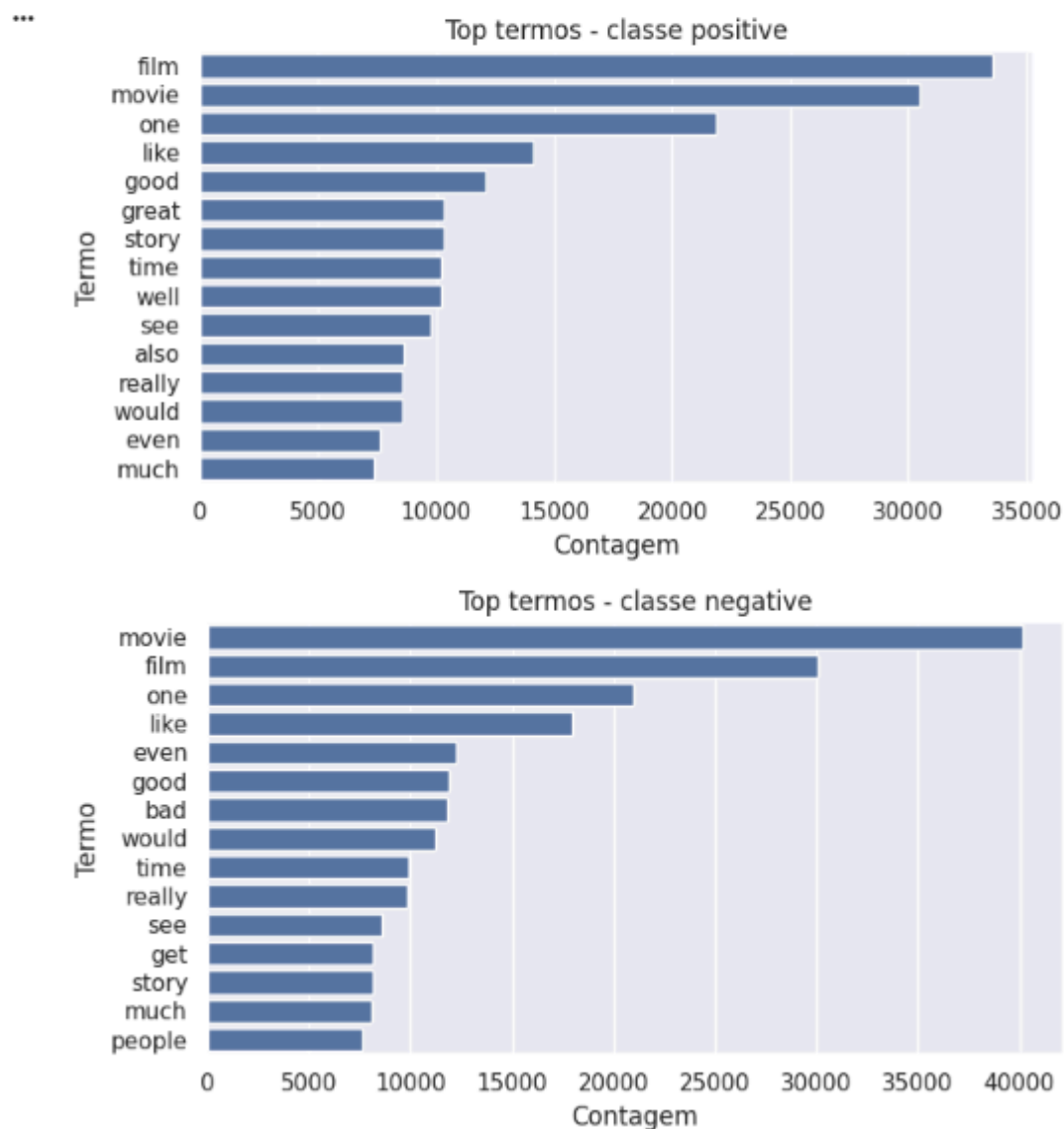
```
...
Top termos (positive):
   term  count
0  film  33575
1  movie  30459
2   one  21849
3  like  14104
4  good  12107

Top termos (negative):
   term  count
0  movie  40147
1   film  30079
2   one  20937
3  like  17936
4  even  12278
```

Imprimimos os gráficos com o top termos mais utilizados para cada classe.

```
# Plots dos top termos
plt.figure(figsize=(7,4))
sns.barplot(data=top_pos, x='count', y='term')
plt.title('Top termos - classe positive')
plt.xlabel('Contagem'); plt.ylabel('Termo')
plt.tight_layout(); plt.savefig('reports/top_terms_positive.png', dpi=150)
plt.show()

plt.figure(figsize=(7,4))
sns.barplot(data=top_neg, x='count', y='term')
plt.title('Top termos - classe negative')
plt.xlabel('Contagem'); plt.ylabel('Termo')
plt.tight_layout(); plt.savefig('reports/top_terms_negative.png', dpi=150)
plt.show()
```



Carregamos o arquivo que geramos no passado com os tratamentos. Fazemos alguns passos para o tratamento de alguma falha.

```
# Carregando o CSV limpo

CSV_CLEAN = '/content/data-processed/imdb_clean.csv'

assert os.path.exists(CSV_CLEAN), f"Arquivo não encontrado: {CSV_CLEAN}"
df_clean = pd.read_csv(CSV_CLEAN)
assert {'text', 'label'}.issubset(df_clean.columns)
print(df_clean.shape, df_clean['label'].value_counts(normalize=True).round(3).to_dict())

... (49582, 2) {'positive': 0.502, 'negative': 0.498}
```

Imprimimos um resumo estatístico do nosso arquivo que importamos.

```
# Resumo estatístico
shape = df_clean.shape
dist = df_clean['label'].value_counts(normalize=True).round(3).to_dict()
print(
    "Dataset limpo carregado:\n"
    f"- Linhas x Colunas: {shape[0]} x {shape[1]}\n"
    f"- Colunas: {list(df_clean.columns)}\n"
    f"- Distribuição de classes: "
    f"positive={dist.get('positive', 0)*100:.2f}% | negative={dist.get('negative', 0)*100:.2f}%"
)

... Dataset limpo carregado:
- Linhas x Colunas: 49582 x 2
- Colunas: ['text', 'label']
- Distribuição de classes: positive=50.20% | negative=49.80%
```

Começamos a fazer a divisão estratificada, e criamos o conjuntos de treinamento com 80% e validação com 20%, utilizando uma semente de 42.

```
# Split estratificado 80/20
from sklearn.model_selection import train_test_split
# Valor de semente
RANDOM_STATE = 42

X_train, X_val, y_train, y_val = train_test_split(
    df_clean['text'], df_clean['label'],
    test_size=0.20, stratify=df_clean['label'], random_state=RANDOM_STATE
)

def dist(series):
    return series.value_counts(normalize=True).sort_index().round(3).to_dict()

print(
    "Split estratificado 80/20 concluído (random_state=42).\n"
    f"- Treino: {len(X_train)} registros | Distribuição: {dist(y_train)}\n"
    f"- Validação: {len(X_val)} registros | Distribuição: {dist(y_val)}"
)

... Split estratificado 80/20 concluído (random_state=42).
- Treino: 39665 registros | Distribuição: {'negative': 0.498, 'positive': 0.502}
- Validação: 9917 registros | Distribuição: {'negative': 0.498, 'positive': 0.502}
```

Começamos a realizar o treinamento e otimizar um modelo de regressão. Criamos o pipeline com TF-IDF, transformando o texto, removemos as palavras comuns definido pelo idioma inglês, configuramos com palavras únicas, definimos no mínimo 5 aparição do termo, modelo de classificação linear, usa L2 ridge para prevenir o overfitting, limitamos em 1000 as interações. Fazemos a validação cruzada. Buscamos a métrica de avaliação. A execução e a saída dos dados.

9) ETAPA

Modelo e Treinamento de Regressão Logística

```
# TF-IDF + Regressão Logística + GridSearch (k=5, F1)
# - Converte os reviews em vetores numéricos com TF-IDF.
# - Treina Regressão Logística (com L2) para classificar positivo/negativo.
# - Usa validação cruzada estratificada (5 folds) e GridSearch para escolher o melhor C via F1.
# - Define F1 com classe positiva explícita para evitar NaN (labels são strings).
# - Devolve o melhor pipeline (vetorizador + modelo) para avaliar no hold-out de 20%.

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.metrics import make_scorer, f1_score

# sanity check opcional (mostra labels do treino)
# esperado: ['negative', 'positive']
print("Labels no treino:", sorted(y_train.unique()))

pipe = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words='english', ngram_range=(1,1), min_df=5)),
    ('clf', LogisticRegression(penalty='l2', max_iter=1000, random_state=RANDOM_STATE))
])

param_grid = {'clf__C': [0.1, 1, 10]}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

# scorer com classe positiva explícita
f1_pos_scorer = make_scorer(f1_score, pos_label='positive')

grid = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    scoring=f1_pos_scorer, # <-- evita NaN
    cv=cv,
    n_jobs=-1,
    verbose=1,
    return_train_score=False
)

grid.fit(X_train, y_train)

print("Melhor C:", grid.best_params_['clf__C'], "| F1 médio (CV):", round(grid.best_score_, 4))

# grava dataframe dos resultados com o scorer
pd.DataFrame(grid.cv_results_).to_csv('reports/cv_results_logreg_tfidf.csv', index=False)
```

```
... Labels no treino: ['negative', 'positive']
Fitting 5 folds for each of 3 candidates, totalling 15 fits
Melhor C: 1 | F1 médio (CV): 0.8931
```

Fazemos a avaliação com Machine Learning avaliando o modelo otimizado no conjunto de dados de validação (hold-out 20%) e gerando as métricas de desempenho e a Matriz de Confusão.

O objetivo é medir o desempenho do melhor pipeline em dados que o modelo nunca viu, garantindo uma estimativa realista da capacidade de generalização do modelo.

Calcula o F1-Score, Precisão e Recall, todos focados na classe 'positive' (conforme definido por `pos_label='positive'`), que geralmente é a classe de interesse primário na análise de sentimentos.

Os resultados dessas métricas são impressos de forma formatada.

Então geramos a matriz de confusão e visualização.

```
# AVALIAÇÃO NO HOLD-OUT (20%) + MATRIZ DE CONFUSÃO
# - Usa o melhor pipeline do GridSearch (TF-IDF + LogReg) para prever no conjunto de validação (20%).
# - Calcula métricas principais para a classe positiva: F1 (principal), Precisão e Recall.
# - Salva um CSV com as métricas em reports/metrics_holdout.csv.
# - Gera e salva a Matriz de Confusão em reports/confusion_matrix.png.
# * Linhas = rótulo real; Colunas = rótulo previsto; ordem: ['negative', 'positive'].

import seaborn as sns, matplotlib.pyplot as plt
from sklearn.metrics import f1_score, precision_score, recall_score, confusion_matrix, classification_report

best_model = grid.best_estimator_
y_pred = best_model.predict(X_val)

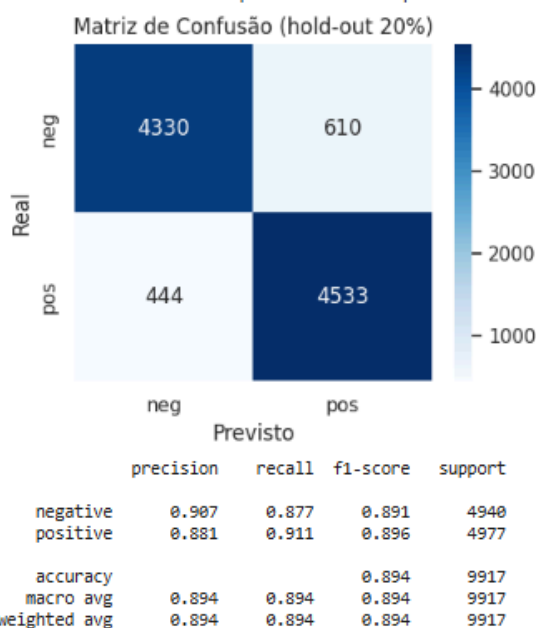
f1 = f1_score(y_val, y_pred, pos_label='positive')
pre = precision_score(y_val, y_pred, pos_label='positive')
rec = recall_score(y_val, y_pred, pos_label='positive')
print(f"Hold-out 20% -> F1={f1:.3f} | Precisão={pre:.3f} | Recall={rec:.3f}")

pd.DataFrame([['f1':f1, 'precision':pre, 'recall':rec]]).to_csv('reports/metrics_holdout.csv', index=False)

cm = confusion_matrix(y_val, y_pred, labels=['negative', 'positive'])
plt.figure(figsize=(4.6,3.8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['neg', 'pos'], yticklabels=['neg', 'pos'])
plt.title('Matriz de Confusão (hold-out 20%)'); plt.xlabel('Previsto'); plt.ylabel('Real')
plt.tight_layout(); plt.savefig('reports/confusion_matrix.png', dpi=150); plt.show()

print(classification_report(y_val, y_pred, digits=3))
```

... Hold-out 20% -> F1=0.896 | Precisão=0.881 | Recall=0.911



O código que você apresentou executa a interpretabilidade do modelo de Regressão Logística treinado, uma etapa crucial para entender como o modelo toma suas decisões. Ele faz isso analisando os coeficientes associados a cada termo (palavra) no vocabulário TF-IDF.

A Regressão Logística, sendo um modelo linear, associa um peso (coeficiente) a cada feature (que aqui são os termos TF-IDF). A magnitude e o sinal desse peso indicam a influência do termo na probabilidade de uma review ser classificada como 'positive'.

Esta visualização é a maneira mais clara de entender o vocabulário do sentimento que o modelo aprendeu. Se a interpretação humana dos termos (ex: "brilliant" é positivo) for coerente com o sinal do coeficiente do modelo (coeficiente > 0), isso aumenta a confiança na validade do classificador.

```
# INTERPRETABILIDADE - Termos mais influentes (coeficientes da Regressão Logística)
# - Extraí o vocabulário do TF-IDF e os coeficientes do classificador (classe 'positive').
# - Coeficiente > 0: termo puxa a predição para POSITIVO; coeficiente < 0: puxa para NEGATIVO.
# - Seleciona os 20 maiores coeficientes (positivos) e os 20 menores (negativos).
# - Salva as tabelas em CSV.
# - Gera gráficos de barras e salva em PNG.

import numpy as np, pandas as pd

tfidf = best_model.named_steps['tfidf']
clf = best_model.named_steps['clf']

terms = np.array(tfidf.get_feature_names_out())
coefs = clf.coef_.ravel()

top_pos_idx = np.argsort(coefs)[-20:][::-1]
top_neg_idx = np.argsort(coefs)[:20]

top_pos = pd.DataFrame({'term': terms[top_pos_idx], 'coef': coefs[top_pos_idx]})
top_neg = pd.DataFrame({'term': terms[top_neg_idx], 'coef': coefs[top_neg_idx]})

top_pos.to_csv('reports/top_coefficients_positive.csv', index=False)
top_neg.to_csv('reports/top_coefficients_negative.csv', index=False)

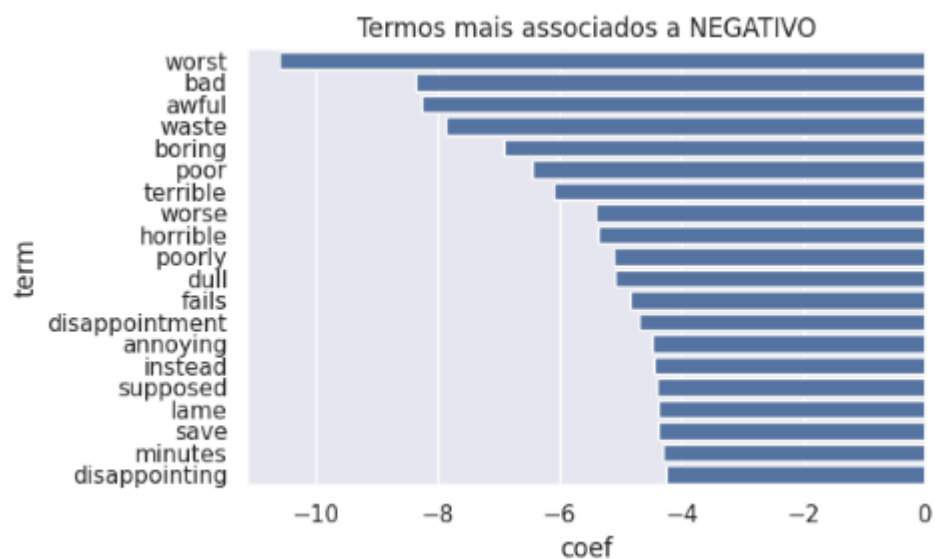
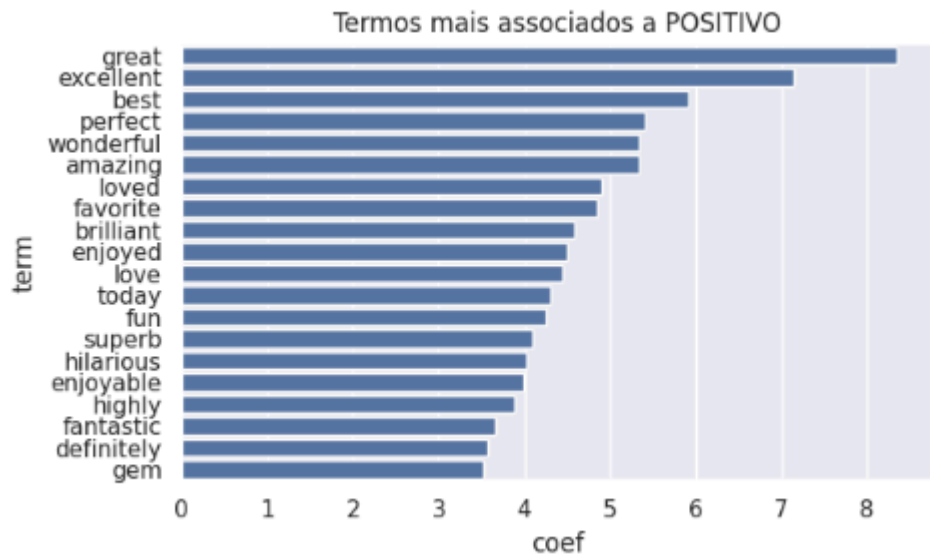
plt.figure(figsize=(6.5,4)); sns.barplot(data=top_pos, x='coef', y='term'); plt.title('Termos mais associados a POSITIVO')
plt.tight_layout(); plt.savefig('reports/top_coefficients_positive.png', dpi=150); plt.show()

plt.figure(figsize=(6.5,4)); sns.barplot(data=top_neg, x='coef', y='term'); plt.title('Termos mais associados a NEGATIVO')
plt.tight_layout(); plt.savefig('reports/top_coefficients_negative.png', dpi=150); plt.show()
```


São gerados dois gráficos de barras separados.

1. **Termos mais associados a POSITIVO:** Barras longas e positivas, mostrando palavras como "excellent", "amazing", "love", etc.
2. **Termos mais associados a NEGATIVO:** Barras longas e negativas, mostrando palavras como "worst", "awful", "boring", etc.

...



O código que você forneceu realiza a Análise de Erros Qualitativa do seu modelo, focando nos casos que ele errou de maneira mais crítica: os Falsos Positivos (FP) e os Falsos Negativos (FN).

Esta é uma etapa fundamental após a avaliação quantitativa, pois permite que o analista de dados entenda os padrões de erros do modelo, o que pode levar a melhorias no pré-processamento, na vetorização (TF-IDF) ou no próprio modelo.

Cria uma coluna auxiliar com os primeiros 500 caracteres do texto. Isso é feito para que os arquivos CSV de relatório sejam mais legíveis, evitando que linhas muito longas (reviews completas) dificultem a inspeção manual.

Salvamento dos Relatórios de Amostra salvando as 10 primeiras amostras de cada tipo de erro nos respectivos arquivos CSV.

Ao inspecionar manualmente esses 10 exemplos, você pode identificar: Uso de ironia/sarcasmo (muitas vezes causa FP ou FN), Duplo Sentido ou negações complexas, Vocabulário não capturado pelo modelo ou pelo TF-IDF, Erros na rotulagem original do dataset (raro, mas possível).

O print final informa o número total de Falsos Positivos e Falsos Negativos no conjunto de validação, fornecendo a contagem exata dos erros que acabaram de ser amostrados.

```
# ANÁLISE DE ERROS - Amostras de FP (falsos positivos) e FN (falsos negativos)
# - Constrói um DataFrame com texto real (X_val), rótulo verdadeiro (label) e previsão (pred).
# - Adiciona a probabilidade do modelo para a classe 'positive' (proba_pos).
# - Separa:
#   * FP: casos com label = 'negative' mas pred = 'positive'
#   * FN: casos com label = 'positive' mas pred = 'negative'
# - Cria versões truncadas dos textos (primeiros 500 caracteres) para facilitar leitura no relatório.
# - Salva 10 exemplos de cada tipo em CSV:
#   * reports/false_positives_sample.csv
#   * reports/false_negatives_sample.csv
# - O print final mostra quantos FP e FN existem no hold-out.

y_pred_proba = best_model.predict_proba(X_val)[:,-1] if hasattr(best_model, "predict_proba") else None

val_df = pd.DataFrame({'text': X_val, 'label': y_val, 'pred': y_pred})
if y_pred_proba is not None:
    val_df['proba_pos'] = y_pred_proba

fp = val_df[(val_df['label']=='negative') & (val_df['pred']=='positive')].copy()
fn = val_df[(val_df['label']=='positive') & (val_df['pred']=='negative')].copy()
fp['text_trunc'] = fp['text'].str.slice(0, 500)
fn['text_trunc'] = fn['text'].str.slice(0, 500)

fp.head(10)[['text_trunc', 'label', 'pred', 'proba_pos']].to_csv('reports/false_positives_sample.csv', index=False)
fn.head(10)[['text_trunc', 'label', 'pred', 'proba_pos']].to_csv('reports/false_negatives_sample.csv', index=False)

print(f"FP: {len(fp)} | FN: {len(fn)} (amostras salvas em reports/*_sample.csv)")

... FP: 610 | FN: 444 (amostras salvas em reports/*_sample.csv)
```

Finaliza o projeto de Machine Learning salvando o modelo treinado e consolidando os resultados mais importantes em um arquivo de resumo. Esta é a etapa de produção e documentação.

O objetivo principal desta fase é garantir que o trabalho realizado (o treinamento do modelo otimizado e a obtenção das métricas) não seja perdido e possa ser facilmente reutilizado ou referenciado no futuro.

A biblioteca joblib é usada para serializar o objeto `best_model`. Como `best_model` é um objeto Pipeline do Scikit-learn, o joblib salva tanto o vetorizador TF-IDF treinado quanto a Regressão Logística otimizada em um único arquivo binário (.pkl).

Benefício: Você pode carregar o arquivo `models/logreg_tfidf_pipeline.pkl` em qualquer outro script e usá-lo imediatamente para prever novos textos, sem precisar reprocessar os dados ou re-treinar o modelo.

```
# Salvar modelo e resumo dos resultados
# - Salva o pipeline completo (TF-IDF + Regressão Logística com melhor C) via joblib,
#   para reutilizar depois sem reprocessar o texto.
# - Gera um resumo em CSV com os principais números: melhor C, F1 no CV (k=5),
#   e métricas do hold-out (F1, Precisão, Recall).
# - Arquivos gerados:
#   * models/logreg_tfidf_pipeline.pkl -> modelo pronto para carregar e prever
#   * reports/summary_stage3.csv      -> tabela com métricas-chave
# - Mantém reprodutibilidade: usamos random_state=42 em split/CV/treino.

import os
os.makedirs('models', exist_ok=True)
os.makedirs('reports', exist_ok=True) # caso ainda não exista

import joblib, pandas as pd

# salvar o modelo
joblib.dump(best_model, 'models/logreg_tfidf_pipeline.pkl')

# salvar resumo
pd.DataFrame([{'best_c': grid.best_params_['clf__C'],
               'cv_f1_mean': round(grid.best_score_, 4),
               'holdout_f1': round(f1, 3),
               'holdout_precision': round(pre, 3),
               'holdout_recall': round(rec, 3)}]).to_csv('reports/summary_stage3.csv', index=False)

print("OK: modelo salvo em models/logreg_tfidf_pipeline.pkl e resumo em reports/summary_stage3.csv")
```

... OK: modelo salvo em models/logreg_tfidf_pipeline.pkl e resumo em reports/summary_stage3.csv

Ao fim das execuções do código encontramos nossos arquivos assim:

